Task 2 – From Requirement to Event-Driven Design: Hospital Outpatient Appointment Desk (8 Marks, K4)

Run each program yourself and attach your own screenshots, each linked to the requirement or test it shows. Items marked `<fill>` need your own observations or measurements.

Task 2.1 – Event model

Event source	Event object	Listener interface	Action taken
Patient ID text field (Enter key)	`ActionEvent`	`ActionListener`	Validate the ID (non-empty, format) and look up the patient name
Patient ID text field (focus lost)	`FocusEvent`	`FocusListener`	Validate the ID and show an inline error
Department `Choice/JComboBox`	`ItemEvent`	`ItemListener`	Refresh the doctor list for that department, clear the slot list
Doctor `Choice/JComboBox`	`ItemEvent`	`ItemListener`	Load the free time slots for that doctor
Time-slot `List/JList`	`ItemEvent` / `ListSelectionEvent`	`ItemListener` / `ListSelectionListener`	Remember the selected slot, enable the Book button
Book button	`ActionEvent`	`ActionListener`	Validate everything, book, show confirmation or error
Clear/Cancel button	`ActionEvent`	`ActionListener`	Reset the form, or ask to confirm if a booking is in progress
Window close button (X)	`WindowEvent`	`WindowListener` / `WindowAdapter` (`windowClosing`)	If a booking is in progress show "Discard booking?" and close only on Yes;

Event source	Event object	Listener interface	Action taken
			otherwise `dispose()`

Task 2.2 – Component hierarchy

```
JFrame  "Outpatient Appointment Desk"            BorderLayout
├── NORTH   JPanel (FlowLayout)                  header
│            └── JLabel  "Appointment Desk"
├── CENTER  JPanel (GridBagLayout)               form
│            ├── JLabel + JTextField   patient ID
│            ├── JLabel + JComboBox    department
│            ├── JLabel + JComboBox    doctor
│            └── JLabel + JList in JScrollPane   time slots
├── SOUTH   JPanel (BorderLayout)
│            ├── NORTH  JPanel (FlowLayout)  ── JButton Book, JButton
Clear
│            └── SOUTH  JLabel               status (confirmation /
error)
```

In plain AWT the equivalents are `Frame`, `Panel`, `Label`, `TextField`, `Choice`, `List`, `Button`.

Task 2.3 – Handler pseudocode

(a) Refresh the doctor list when the department changes

```
PROCEDURE onDepartmentChanged(event)
    IF event.stateChange is not SELECTED THEN RETURN          //
ignore the DESELECTED event
    dept  ← departmentBox.selectedItem
    doctorBox.removeAllItems()
    slotList.clear()
    IF dept is null THEN RETURN
    doctors ← doctorMap.get(dept)                              //
Map<String, List<Doctor>>, loaded once
    FOR EACH d IN doctors DO
        doctorBox.addItem(d.name)
    END FOR
    bookButton.disable()
END PROCEDURE
```

(b) Validate and book the appointment

```
PROCEDURE onBookClicked(event)
    id   ← patientField.text.trim()
    dept ← departmentBox.selectedItem
    doc  ← doctorBox.selectedItem
    slot ← slotList.selectedValue
    IF id is empty OR id does not match ID-pattern THEN
        showStatus("Enter a valid patient ID"); RETURN
    IF dept is null OR doc is null THEN
```

```
        showStatus("Select department and doctor"); RETURN
    IF slot is null THEN
        showStatus("Select a time slot"); RETURN
    bookingInProgress ← true ; bookButton.disable()
    RUN IN BACKGROUND (SwingWorker)
        IF slot is no longer free THEN result ← ERROR "Slot already
taken"
        ELSE save appointment ; result ← OK "Booked with Dr. <doc> at
<slot>"
    ON COMPLETION (back on the event-dispatch thread)
        showStatus(result.message)
        bookingInProgress ← false ; bookButton.enable()
END PROCEDURE
```

Task 2.4 – Event trace (open to close)

```
main()
 └ new Frame(...) / new JFrame(...)          constructor builds the
components
 └ add(...) components, addWindowListener(...), registers listeners
 └ setSize(...)/pack(), setVisible(true)
     ├ windowOpened(e)            (first time the window is shown)
     ├ windowActivated(e)         window gets focus
     ├ paint(g) / paintComponent  (and again on every repaint)
user interacts: itemStateChanged, actionPerformed, focusGained/Lost,
mouse/key events ...
user minimises:  windowIconified(e) , windowDeactivated(e)
user restores:   windowDeiconified(e), windowActivated(e), paint(g)
user clicks X:   windowClosing(e)   → handler asks confirmation, then
dispose()
dispose():       windowClosed(e)    → resources released, JVM ends
when no windows remain
```

Comparison with an applet (`init, start, paint, stop, destroy`):

Frame/JFrame window	Applet	Meaning
constructor	`init()`	one-time setup of components
`setVisible(true)` / `windowOpened`, `windowDeiconified`	`start()`	begin or resume activity
`paint()`	`paint()`	draw the screen
`windowIconified`	`stop()`	pause activity when not visible
`windowClosing` / `windowClosed` / `dispose()`	`destroy()`	release resources

An applet's life cycle is controlled by the browser; a frame's life cycle is controlled by the program and the window manager, through `WindowEvent`.

Alternative concept and complexity

Fetch doctors from the database on every department change vs. a preloaded `Map<String, List<Doctor>>`:

```
// Preloaded once (at start-up or in a SwingWorker)
Map<String, List<Doctor>> byDept = loadAll();          // d
departments, m doctors in total
// on change:   doctorBox  ← byDept.get(dept)           O(doctors in
that department)

// Per change:  SELECT name FROM doctors WHERE dept = ?   run in a
SwingWorker
```

- **Time per department change:** map lookup is O(1) plus O(m/d) average to fill the combo box, with no waiting. A database query adds network and disk latency (milliseconds to seconds), roughly O(log m) with an index plus the round trip, and must be on a background thread or the window freezes.
- **Memory:** the map holds all m doctors and d keys: O(d + m), loaded once. The per-change approach holds only one department's doctors at a time, O(m/d), but creates new result sets and objects on every change (more garbage and more load on the database).
- **Trade-off:** the map is faster and simpler for a small, rarely changing list, but can be stale if doctors change. The query is always fresh but slower. A cache with expiry combines both. Own measurement `<fill>`.